# LAB 20.1 AIAC

## 2303A51494

## B27

## Task 1 – Input Validation Check

## PROMPT:

generate a simple username and password login program to review whether input sanitation and validation are implemented also suggest secure improvements

## CODE:

LAB20-1.PY > validate_password

```python
#generate a simple username and password login program to review whether input sanitation and validation are implemented
#also suggest secure improvements
Windsurf: Refactor | Explain | Generate Docstring | ×
def validate_username(username):
    if len(username) < 5:
        return False, "Username must be at least 5 characters long."
    if not username.isalnum():
        return False, "Username must be alphanumeric."
    return True, "Username is valid."
Windsurf: Refactor | Explain | Generate Docstring | ×
def validate_password(password):
    if len(password) < 8:
        return False, "Password must be at least 8 characters long."
    if not any(char.isupper() for char in password):
        return False, "Password must contain at least one uppercase letter."
    if not any(char.islower() for char in password):
        return False, "Password must contain at least one lowercase letter."
    if not any(char.isdigit() for char in password):
        return False, "Password must contain at least one digit."
    if not any(char in "!@#$%^&*()-_=+[]{}|;:'\",.<>?/" for char in password):
        return False, "Password must contain at least one special character."
    return True, "Password is valid."
Windsurf: Refactor | Explain | Generate Docstring | ×
def main():
    username = input("Enter your username: ")
    is_valid_username, username_message = validate_username(username)
    if not is_valid_username:
        print(username_message)
        return
```

```python
def validate_password(password):
        return False, "Password must contain at least one lowercase letter."
    if not any(char.isdigit() for char in password):
        return False, "Password must contain at least one digit."
    if not any(char in "!@#$%^&*()-_=+[]{}|;:'\",.<>?/" for char in password):
        return False, "Password must contain at least one special character."
    return True, "Password is valid."
Windsurf: Refactor | Explain | Generate Docstring | X
def main():
    username = input("Enter your username: ")
    is_valid_username, username_message = validate_username(username)
    if not is_valid_username:
        print(username_message)
        return
    password = input("Enter your password: ")
    is_valid_password, password_message = validate_password(password)
    if not is_valid_password:
        print(password_message)
        return
    print("Login successful!")
if __name__ == "__main__":
    main()
```

# OUTPUT:

```
PROBLEMS 1K+   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
TERMINAL
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> & C:/Users/Pavani/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Pavani/OneDrive/Documents/AI Assisted coding/LAB20-1.PY"
Enter your username: Pavani
Enter your password: Pavani@1023
Login successful!
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> & C:/Users/Pavani/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Pavani/OneDrive/Documents/AI Assisted coding/LAB20-1.PY"
Enter your username: Pavani
Enter your password: pavani123
Password must contain at least one uppercase letter.
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding>
```

# EXPLANATION:

- The program verifies that the username and password meet basic validation rules before allowing access.
- It ensures the username is long enough and contains only valid characters.
- It enforces a strong password by requiring a mix of letters, numbers, and special symbols.
- If any input is invalid, the login is stopped to maintain security

# Task 2 – SQL Injection Prevention

## PROMPT:

generate a python script using sqlite/mysql to fetch user details and identify if the code is vulnerable to sql injection and refactor using parameterized queries

## CODE:

```python
#generate a python script using sqlite/mysql to fetch user details and
#identify if the code is vulnerable to sql injection and refactor using parameterized queries
import sqlite3
Windsurf: Refactor | Explain | Generate Docstring |
def fetch_user_details(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # Vulnerable to SQL injection
    # query = f"SELECT * FROM users WHERE username = '{username}'"
    # cursor.execute(query)

    # Refactored using parameterized queries
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (username,))

    user_details = cursor.fetchone()
    conn.close()

    return user_details
# Example usage
if __name__ == "__main__":
    username_input = input("Enter username to fetch details: ")
    details = fetch_user_details(username_input)
    if details:
        print("User Details:", details)
    else:
        print("User not found.")
```

## OUTPUT:

```
sqlite3.OperationalError: no such table: users
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> python setup.py
Database and table setup completed successfully!
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> python app.py
Enter username to fetch details: pavani
User Details: (1, 'pavani', 'Pass@123')
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding>
```

## EXPLANATION:

- The setup file creates a database and a users table to store login data.
- It inserts a sample user so the program can retrieve data for testing.
- The main program takes a username as input and searches it in the database.
- It uses a secure query method to prevent SQL injection and ensure safe data access.

# Task 3 – Cross-Site Scripting (XSS) Check

## PROMPT:

generate a feedback form with javascript based output and test whether untrusted inputs are directly rendered without escaping and implement secure measures eg. escaping html entities using csp

## CODE:

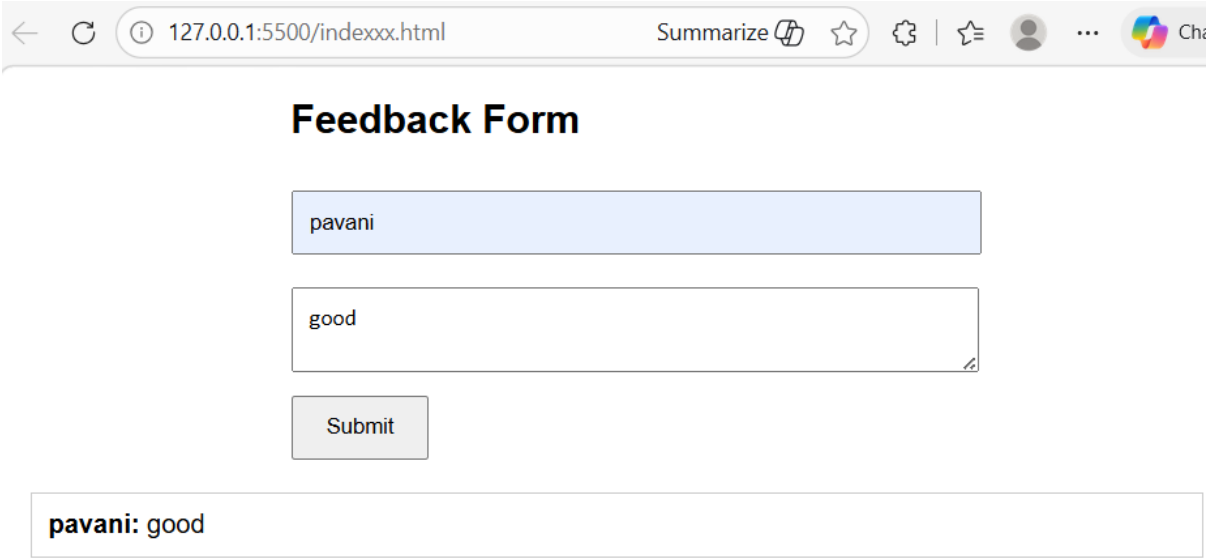
```html
<!-- generate a feedback form with javascript based output and test whether untrusted inputs are directly rendered without escaping and implement s
-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Feedback Form</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .feedback-form {
            max-width: 400px;
            margin: auto;
        }
        .feedback-form input, .feedback-form textarea {
            width: 100%;
            padding: 10px;
            margin: 10px 0;
        }
        .feedback-form button {
            padding: 10px 20px;
        }
        .feedback-output {
            margin-top: 20px;
            padding: 10px;
            border: 1px solid #ccc;
```

indexxx.html > ...

```html
<html lang="en">
<head>
    <style>
        }
    </style>
</head>
<body>
    <div class="feedback-form">
        <h2>Feedback Form</h2>
        <input type="text" id="name" placeholder="Your Name">
        <textarea id="feedback" placeholder="Your Feedback"></textarea>
        <button onclick="submitFeedback()">Submit</button>
    </div>
    <div class="feedback-output" id="feedbackOutput"></div>

    <script>
        Windsurf: Refactor | Explain | Generate Function Comment | ×
        function escapeHtml(unsafe) {
            return unsafe
                .replace(/&/g, "&amp;")
                .replace(/</g, "&lt;")
                .replace(/>/g, "&gt;")
                .replace(/"/g, "&quot;")
                .replace(/'/g, "&#039;");
        }

        Windsurf: Refactor | Explain | Generate Function Comment | ×
        function submitFeedback() {
            const name = document.getElementById('name').value;
            const feedback = document.getElementById('feedback').value;
```

```html
<body>
    <script>
        function escapeHtml(unsafe) {
                .replace(/&/g, "&amp;")
                .replace(/</g, "&lt;")
                .replace(/>/g, "&gt;")
                .replace(/"/g, "&quot;")
                .replace(/'/g, "&#039;");
        }

        Windsurf: Refactor | Explain | Generate Function Comment | ×
        function submitFeedback() {
            const name = document.getElementById('name').value;
            const feedback = document.getElementById('feedback').value;

            // Escape user inputs to prevent XSS
            const escapedName = escapeHtml(name);
            const escapedFeedback = escapeHtml(feedback);

            // Display the feedback safely
            const feedbackOutput = document.getElementById('feedbackOutput');
            feedbackOutput.innerHTML = `<strong>${escapedName}:</strong> ${escapedFeedback}`;
        }
    </script>
</body>
</html>
```

## OUTPUT:




## EXPLANATION:

- The program collects user feedback through a simple HTML form with name and message fields.
- It uses JavaScript to display the entered data dynamically on the page after submission.
- To prevent security issues like XSS, user inputs are escaped using a function before rendering.
- This ensures untrusted input is safely displayed without executing harmful scripts

**Task 4 – Real-Time Application: Security Audit of AI-Generated Code**

**PROMPT:**

select the code snippet eg login form and perform a security audit to detect possible vulnerabilities and suggest secure coding practices to fix issues and compare insecure vs secure versions side by side

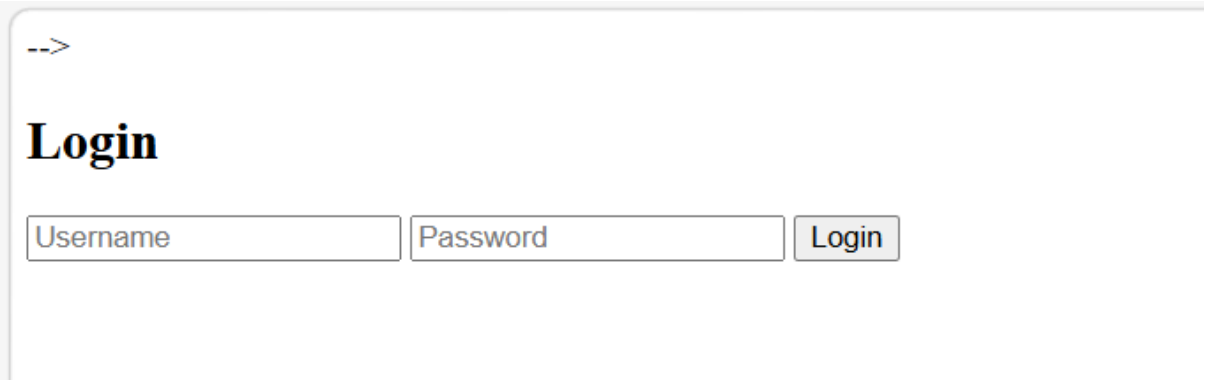
# CODE:

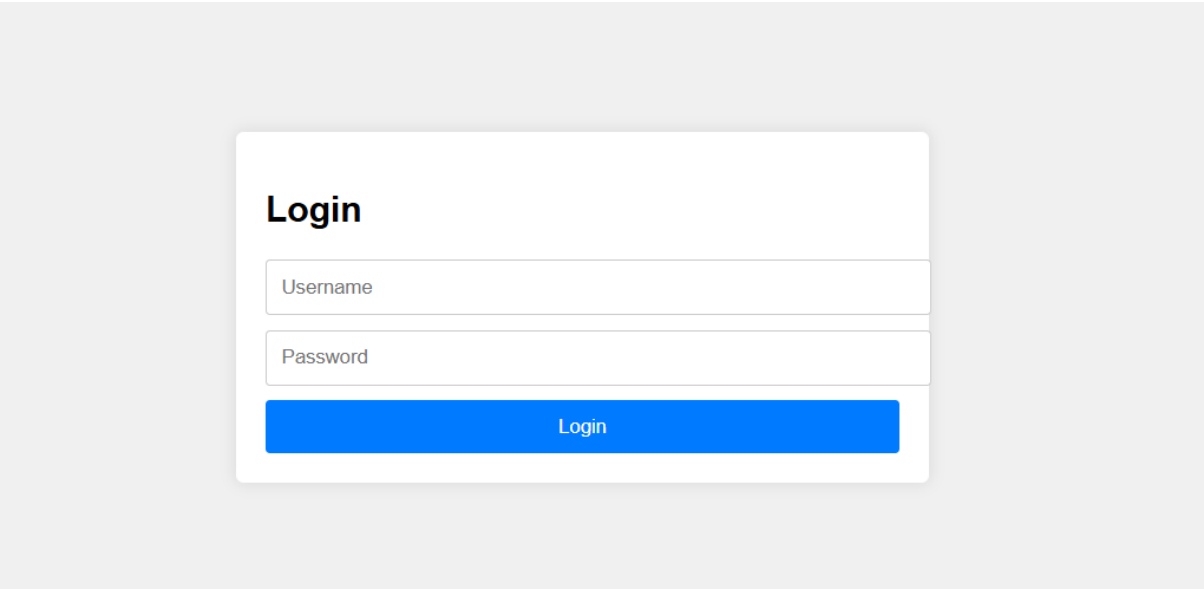
```html
<!-- select the code snippet eg login form and perform a security audit to detect possible vulnerabilities and suggest secure coding practi
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login Form</title>

    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f0f0f0;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
        }
        .login-container {
            background-color: #fff;
            padding: 20px;
            border-radius: 5px;
            box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
        }
        .login-container h2 {
            margin-bottom: 20px;
        }
        .login-container input {
            width: 100%;
```

```html
        }
        .login-container button {
            width: 100%;
            padding: 10px;
            background-color: #007BFF;
            color: #fff;
            border: none;
            border-radius: 3px;
            cursor: pointer;
        }
        .login-container button:hover {
            background-color: #0056b3;
        }
    </style>
</head>
<body>
    <div class="login-container">
        <h2>Login</h2>
        <form action="/login" method="POST">
            <input type="text" name="username" placeholder="Username" required>
            <input type="password" name="password" placeholder="Password" required>
            <button type="submit">Login</button>
        </form>
    </div>
</body>
</html>
```

```html
<!-- insecure version  -->
<!DOCTYPE html>
<html lang="en"></html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login Form</title>
</head>
<body>
    <h2>Login</h2>
    <form action="/login" method="POST">
        <input type="text" name="username" placeholder="Username" required>
        <input type="password" name="password" placeholder="Password" required>
        <button type="submit">Login</button>
    </form>
</body>
</html>
```

**OUTPUT:**






**EXPLANATION**

- The task shows both an insecure and a secure version of a basic login form in HTML.
- The insecure version simply takes input without any protection, which can lead to security risks.
- The secure version represents a safer approach, where such forms are expected to be combined with validation and backend security measures.
- It highlights the importance of handling user input carefully to prevent issues like data misuse or attacks.

# Task 5 – Insecure Data Serialization Check

## PROMPT:

generate a script using pickle.load() and explain why untrusted pickle data is dangerous and replace with safer formats like json

## CODE:

{} trusted_data.json > ...

```
{
    "name": "Pavani",
    "age": 20,
    "course": "CSE"
}
```

```
#generate a script using pickle.load() and explain why untrusted pickle data is dangerous and replace with safer formats like json
import pickle
import json
# Example of loading untrusted pickle data (dangerous)
# with open('untrusted_data.pkl', 'rb') as file:
#     data = pickle.load(file)
#     print("Data loaded from pickle:", data)
# Explanation: Untrusted pickle data is dangerous because it can execute arbitrary code during the unpickling process. If an attacker crafts a malic
# Safer alternative using JSON
Windsurf: Refactor | Explain | Generate Docstring | X
def load_json_data(file_path):
    with open(file_path, 'r') as file:
        data = json.load(file)
        print("Data loaded from JSON:", data)
# Example usage
if __name__ == "__main__":
    load_json_data('trusted_data.json')
```

## OUTPUT:

```
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> & C:/Users/Pavan
ents/AI Assisted coding/LAB20-1.PY"
Data loaded from JSON: {'name': 'Pavani', 'age': 20, 'course': 'CSE'}
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding>
```

**EXPLANATION:**

- The program reads data from a JSON file using Python's json module.
- It opens the file in read mode and loads the content into a Python object.
- The loaded data is then printed to verify the file contents.
- It demonstrates safe handling of trusted input files for data processing.